Senior Project Final Report

Creative Team

Members

Alexander Boyce - 1485680@uhv.edu

Edward Cruz - 1773702@uhv.edu

Nicholas Rodriguez - 1077499@uhv.edu

Pooria Zare - 1941982@uhv.edu

Advisor

Wenjuan Huang - huangw1@uhv.edu

Submitted on

May 6, 2025

Abstract

In the field of supply chain logistics, businesses face significant challenges in processing and managing large volumes of printed documents such as invoices, dock receipts, and packing lists. Manual data extraction from these documents is time-consuming, error-prone, and creates bottlenecks across the supply chain workflow. There is a growing need for a solution that can efficiently and accurately automate the extraction of key information from printed PDFs and image files, without relying on costly and inflexible manual processes.

To address this problem, our project integrates existing pre-trained machine learning models and optical character recognition (OCR) technologies to automate document processing. Leveraging tools such as Tesseract OCR with Python integration via PyTesseract, and image handling through the Pillow library, the system extracts structured data from scanned PDFs and image files. This approach focuses solely on printed documents, excluding handwritten inputs to maximize processing accuracy and system reliability. The solution connects to a lightweight, portable database backend using SQLite for efficient document metadata management. Key results thus far include the successful development of file upload and document preview functionalities, OCR pipeline setup, and database integration. Our work demonstrates that by utilizing existing AI technologies and streamlining integration workflows, it is possible to significantly reduce administrative overhead and enhance operational efficiency in logistics.

Keywords: Optical Character Recognition (OCR), Pre-trained AI Models, Tesseract, Document Automation, Supply Chain Logistics, Printed Document Processing, PDF and Image Handling, Data Extraction.

Introduction

Problem Statement

In the logistics and supply chain industry, the processing of large volumes of printed documents, such as invoices, packing lists, dock receipts, and bills of lading, is a major administrative burden. Traditionally, human workers manually extract key data from these documents and input it into proprietary business systems. This method is slow, error-prone, and consumes valuable operational resources, creating inefficiencies that ripple throughout the supply chain. Despite advances in digitalization, many small and mid-sized Suppliers and customers still provide documents in varying printed formats, making full system integration difficult. Automation in supply chain logistics has been shown to increase operational efficiency,

reduce labor costs, and enhance data accuracy. Therefore, there is a significant need for a system that can automate document data extraction without requiring intensive manual oversight or complex system overhauls.

Solution Overview

Our project addresses this need by developing a document processing system that utilizes existing pre-trained optical character recognition (OCR) and machine learning (ML) technologies to automate the extraction of structured data from printed PDFs and image files. We have integrated tools such as Py-Tesseract, an OCR library based on Google's Tesseract OCR, combined with the Python Pillow package to efficiently handle PDFs by converting them to images for processing. Rather than training a new model from scratch, our system leverages established technologies to rapidly deploy a scalable and adaptable solution. The extracted information is organized into structured outputs supported by a lightweight, reliable SQLite database for storage and retrieval. Our focus on excluding handwritten documents ensures higher initial accuracy and faster implementation. The project also includes a user-friendly front-end where users can upload and preview documents, and a secure backend that processes and stores the extracted data.

Contribution to Existing Work

This project builds upon and extends existing OCR and machine learning technologies by combining them into a practical, deployable system specifically tailored for supply chain logistics document processing. While OCR solutions like Tesseract have been applied to general text recognition tasks, our work uniquely adapts and integrates these tools for the logistics environment, optimizing the system for the specific formats and requirements of printed supply chain documents. Furthermore, by focusing on modularity and lightweight database solutions, we create a system that can be easily maintained, updated, and scaled without heavy computational costs or the need for constant retraining. In doing so, our project demonstrates how accessible, pre-trained AI tools can solve real-world administrative problems efficiently and cost-effectively, offering a practical roadmap for organizations seeking to modernize their supply chain operations.

Literature Survey

Overview of Related Research and Existing Tools

Optical Character Recognition (OCR) and machine learning have long been explored to automate document processing tasks in industries like logistics. Common solutions include

open-source engines such as **Tesseract OCR**, commercial cloud-based tools like **Google Cloud Vision OCR** and **Amazon Textract**, and advanced techniques like **Retrieval-Augmented Generation (RAG)**.

Tesseract OCR, maintained by Google, is a reliable open-source engine for printed text recognition, ideal for structured documents like invoices or packing slips [1]. For preprocessing and handling of PDF files, tools like **Pillow** and **pdf2image** libraries are widely used to convert documents into image formats suitable for Tesseract.

Cloud platforms such as **Google Cloud Vision OCR** and **Amazon Textract** offer broader format support and handwriting recognition but raise privacy and cost concerns when handling proprietary or sensitive documents. **ABBYY FineReader** offers high accuracy and layout awareness but comes with licensing fees and a desktop-oriented workflow [4].

In addition to OCR-specific tools, the team integrated pre-trained large language models via **Cohere's Secure AI Platform**, which provides NLP capabilities for extracting contextually relevant information. The **WebUI** project on GitHub enabled the use of browser-based front-end interfaces through Python backends, providing a lightweight and responsive GUI framework for user interactions [2].

For UI/UX development and front-end formatting, **MDN Web Docs** served as the primary reference for HTML/CSS behaviors, while **Bootstrap** enabled rapid interface design and responsive layouts. **SQLite** was chosen as the embedded database due to its simplicity, portability, and ease of integration into lightweight applications, replacing more complex solutions like PostgreSQL for this project.

Comparison of Major OCR Tools

Tool	Strengths	Limitations	Suitability for Our Project
Tesseract OCR	Open-source, customizable, good for printed text	Poor with handwriting, needs preprocessing	Highly suitable
Google Cloud Vision OCR	High accuracy, cloud-based NLP support	Internet required, cost, data privacy	Not preferred
Amazon Textract	Table extraction, key-value recognition	Expensive, AWS lock-in	Not suitable
ABBYY FineReader	High precision, layout awareness	Paid license, less scalable	Not suitable
RAG Frameworks	Great for unstructured data	Too complex for our use case	Overkill

Comparison of Tools Used in Our Codebase

Tool/Resource	Role in Project	Justification for Use
Tesseract OCR	Core OCR engine for printed document extraction	Free, accurate for structured text
Pillow/pdf2image	Convert PDF to image format for OCR	Essential for preprocessing PDFs
Cohere	Pre-trained NLP for future data enrichment & reasoning	No training needed, enterprise-ready APIs
WebUI	Lightweight browser-based UI frontend	Easy integration with Python backend
SQLite	Embedded database for document metadata and OCR results	Simple, portable, avoids heavy DB overhead
Bootstrap	Front-end framework for UI styling and responsiveness	Quick development and clean design
MDN Web Docs	Documentation for HTML, CSS, JavaScript	Trusted developer resource

Summary

Rather than building a machine learning model from scratch, our team carefully selected a combination of powerful open-source and commercial-grade tools that enable scalable, maintainable document processing for supply chain logistics. By integrating **Tesseract OCR** with **Pillow**, **WebUI**, and **SQLite**, and referencing development standards from **MDN** and **Bootstrap**, we created a cost-effective system without compromising accuracy or performance. Further integration with **Cohere** demonstrates how commercial AI services can complement traditional OCR methods, especially for tasks that may involve future natural language understanding or entity classification.

Citations

1. Smith, R. (2007). *An Overview of the Tesseract OCR Engine*. Proceedings of the Ninth International Conference on Document Analysis and Recognition (ICDAR 2007). https://doi.org/10.1109/ICDAR.2007.4376991
2. WebUI Project. (n.d.). *GitHub - webui-dev/python-webui*. Retrieved from: https://github.com/webui-dev/python-webui

3. Google Cloud. (n.d.). *Cloud Vision API Documentation*. Retrieved from: https://cloud.google.com/vision
4. ABBYY. (n.d.). *FineReader PDF*. Retrieved from: https://pdf.abbyy.com
5. Lewis, P., et al. (2020). *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*. arXiv preprint arXiv:2005.11401. https://arxiv.org/abs/2005.11401
6. Cohere. (2025). *The Secure AI Platform for Enterprise*. Retrieved from: https://cohere.com
7. Mozilla Developer Network (MDN). (n.d.). *MDN Web Docs*. Retrieved from: https://developer.mozilla.org
8. SQLite Consortium. (n.d.). *SQLite Documentation*. Retrieved from: https://www.sqlite.org/docs.html
9. Python Software Foundation. (n.d.). *Pillow (PIL Fork) Documentation*. Retrieved from: https://pillow.readthedocs.io/
10. GitHub - Belval. (n.d.). *pdf2image Documentation*. Retrieved from: https://github.com/Belval/pdf2image

System Design

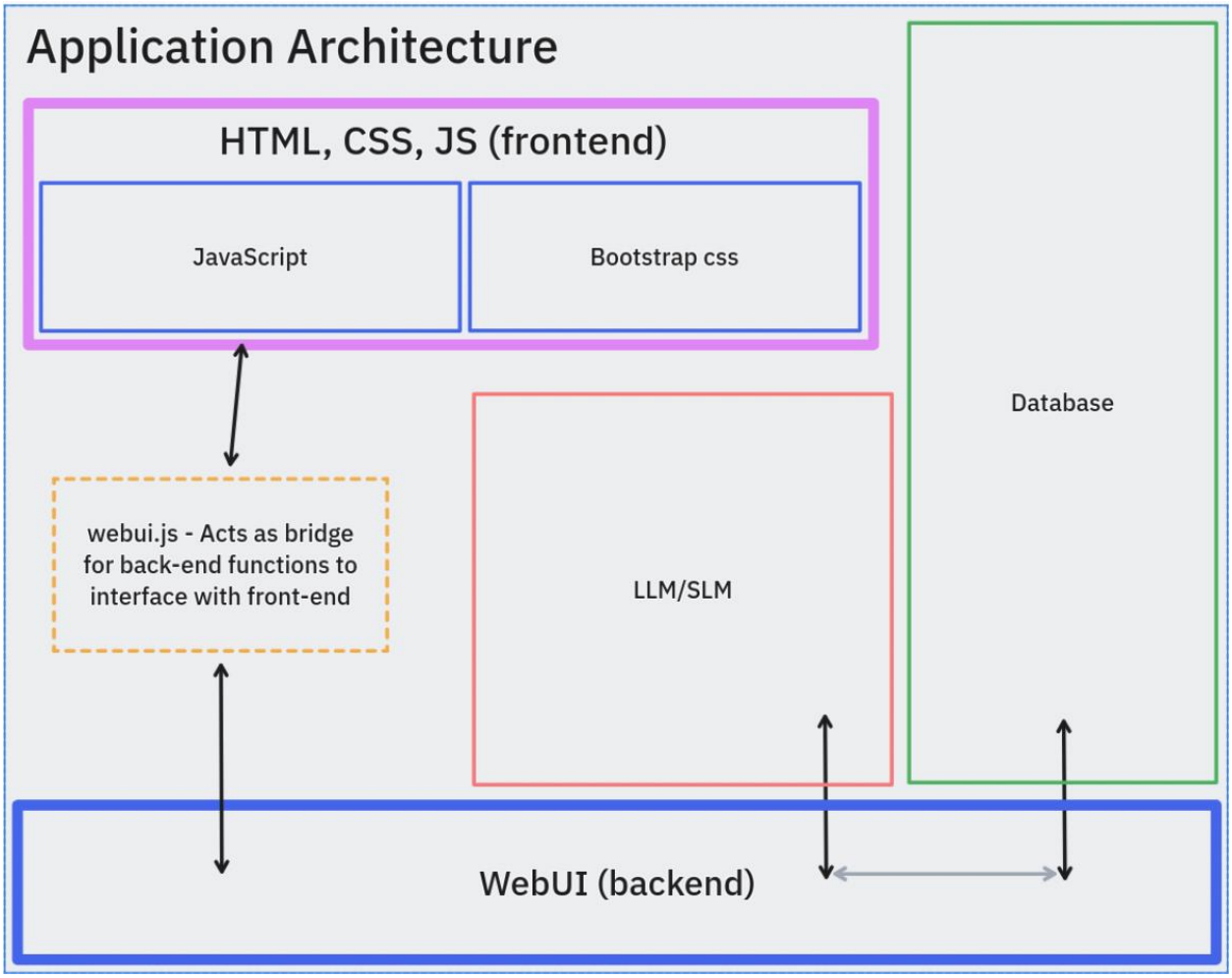
Overview of System Architecture

Our system is designed as a modular pipeline that automates the extraction and structure of printed document data from PDF and image files using OCR. The processed data is stored in a local database for future retrieval or integration where we then send out to the frontend as structured JSON. The system is composed of the following core components:

- **User Interface (Frontend)**
- **Backend Server & API Layer**
- **OCR Processing Engine**
- **PDF/Image Conversion Layer**
- **Database (SQLite)**

The high-level flow is as follows:

1. The user uploads documents via the web interface.
2. The backend receives the files, preprocesses them (e.g., PDF → image).
3. OCR is applied to the image using Tesseract.
4. Extracted text is cleaned and parsed using basic rules or NLP logic.
5. Data is saved to SQLite to then be used for retrieval whenever.



Document Upload and Preprocessing

Frontend Upload Flow

The frontend, built using **WebUI**, allows users to drag and drop PDF, PNG, and JPG files into a dynamic upload area. A preview card UI shows thumbnails and upload history.

File Conversion Logic

Since **Tesseract** does not natively support PDF files, we preprocess PDFs using:

- **pdf2image** (converts PDF to high-resolution PNG images)

- **Pillow** (handles byte-level image operations, resizing, and formatting)

Pseudocode for PDF conversion:

```python
from pdf2image import convert_from_bytes
from PIL import Image

def convert_pdf_to_images(pdf_bytes):
    images = convert_from_bytes(pdf_bytes, dpi=300)
    return [img.convert('RGB') for img in images]
```

This ensures that printed documents are converted into OCR-readable image files before analysis.

OCR Processing and Text Extraction

We use **Tesseract OCR** (via PyTesseract) to extract text from images. It is configured with the --psm (page segmentation mode) set to auto-detect layouts for more accurate parsing of structured documents like invoices and receipts.

```python
import pytesseract

def extract_text(image):
    return pytesseract.image_to_string(image, config='--psm 6')
```

To clean the raw text, we apply basic string parsing:

- Line-by-line splitting
- Regex to detect dates, amounts, and identifiers
- Optional NLP taggers for improved context handling (planned for Cohere)

Example Output:

Invoice Number: 13452
Date: 2025-04-01
Total: $2,352.50

Backend and Data Storage Design

The system's database layer is implemented using SQLite due to its portability, simplicity, and zero-configuration deployment. SQLite is embedded into the application, eliminating the need for additional server installations and making it an ideal choice for local testing, development, and constrained enterprise environments.

Schema example:

```sql
CREATE TABLE IF NOT EXISTS docs (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        data_url TEXT,
        message TEXT,
        chat_history_id INTEGER,
        chat_id INTEGER,
        FOREIGN KEY (chat_history_id) REFERENCES chat_history(id) ON DELETE
CASCADE,
        FOREIGN KEY (chat_id) REFERENCES chat(id)
    );
```

The db.py script encapsulates all interactions with the SQLite database and provides typed, structured functions to insert, query, update, or delete data across five core entities: chat_history, chat, role, docs, and cohere_key. Each table is purpose-built:

- chat_history: Tracks separate conversation sessions.
- chat: Stores individual messages from either the user or assistant.
- role: Defines role data.
- docs: Stores uploaded documents (as base64-encoded URLs) and their extracted OCR content.
- cohere_key: Manages the currently active Cohere API key.

The database schema is designed to support three core components of the document processing workflow:

1. **Chat Sessions and History**
   Each user session is uniquely identified and tracked using a chats table. Messages within a session are stored in the chat table, linked to their parent chat via a foreign key. This structure enables retrieval of full conversations and supports role-based chat segregation (e.g., user, assistant).
2. **Document Storage**
   Scanned documents are stored locally, and their associated document URLs are stored in the docs table. This table includes foreign key references to related chats, enabling seamless document-to-chat linking for context-aware conversations.
3. **Roles and API Key Management**
   To support multi-role interactions (e.g., User, Assistant), a roles table stores predefined role titles. Additionally, the Cohere API key is stored in a dedicated cohere table, allowing dynamic retrieval without hardcoding sensitive credentials.

Modular Components and Extensibility

Each part of the system is modular and designed to be replaced or upgraded in the future:

Component	Current Tool	Extensible To
OCR Engine	Tesseract	Google Vision, Textract
PDF Handling	pdf2image + Pillow	PyMuPDF, Poppler
DB	SQLite	PostgreSQL, MongoDB
Frontend	WebUI + Bootstrap	React, Vue, or Electron
NLP Integration	Planned: Cohere	OpenAI, SpaCy, HuggingFace

This design ensures the system is adaptable — for instance, switching to cloud OCR services or introducing handwriting recognition could be done without a complete rewrite.

Sample Workflow

Step-by-Step Use Case:

1. User visits web app and uploads invoice.pdf
2. Backend converts PDF to PNG and extracts text via OCR
3. Text is cleaned, parsed, and saved
4. User views extracted data in UI

5. Optional: Post-process with NLP for field tagging (future extension)

Summary of Design Decisions

- **Simplicity and speed**: Local-first approach using lightweight libraries.
- **Scalability**: Modular architecture allows component upgrades.
- **Security**: No third-party cloud processing — full control over data.
- **Accuracy**: Focused on printed text for high OCR reliability.

Implementation and Experimental Results

Overview of Implementation

The project was implemented using a Python-based tech stack, with integration of open-source libraries and tools for OCR, file conversion, front-end display, and database management. Our goal was to build a modular, testable, and secure document processing system focused on printed PDFs and images relevant to supply chain logistics.

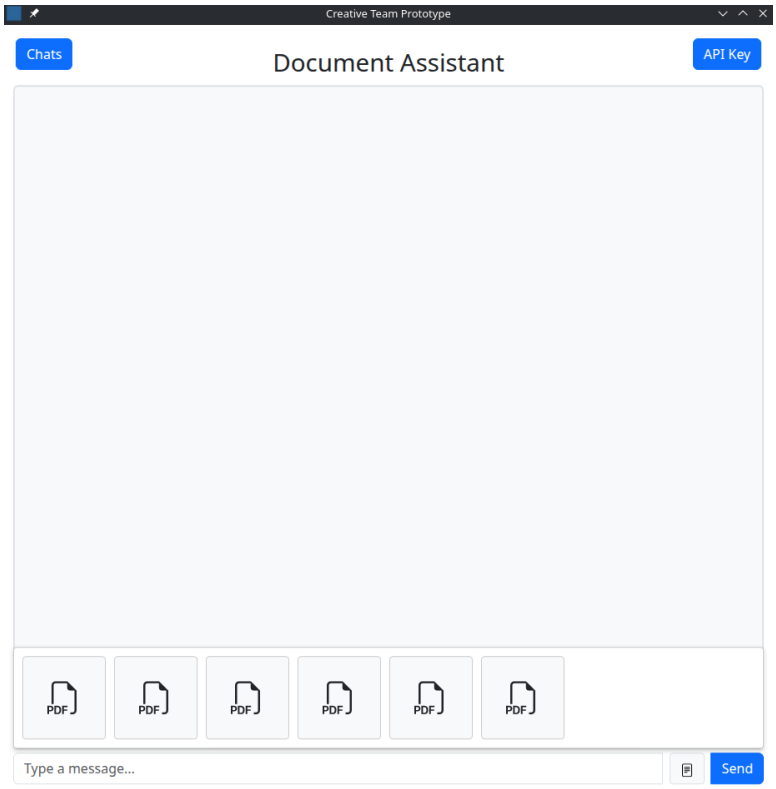
Technologies Used:

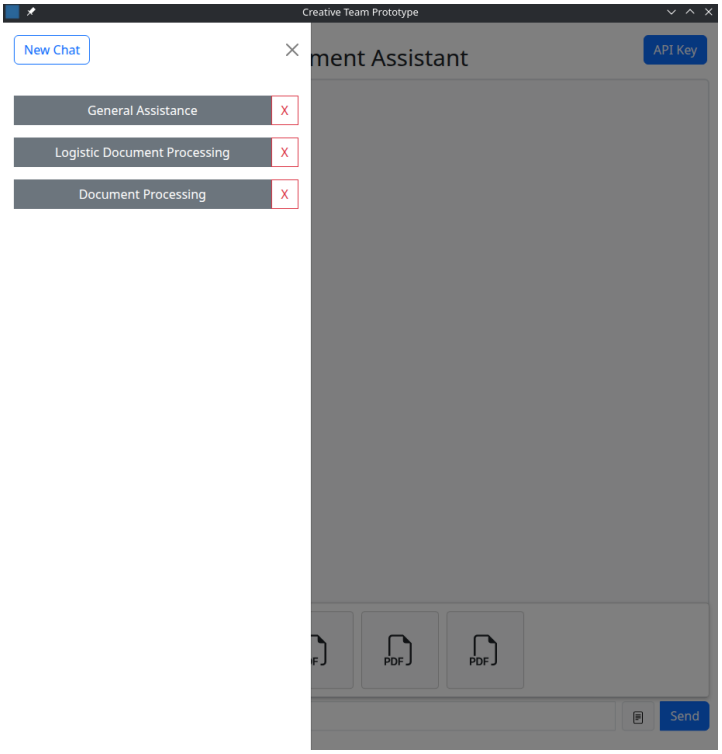
- **Backend**: Python, WebUI-Python, SQLite
- **Frontend**: HTML, Bootstrap CSS Library, JavaScript
- **OCR**: Tesseract OCR via PyTesseract
- **PDF Handling**: pdf2image and Pillow
- **File types supported**: .pdf, .jpg, .png

Front-End Functionality

The front-end was built using **WebUI** with Bootstrap styling to allow simple and clean file uploads and document previews. Users can:

- Upload multiple files
- See preview thumbnails with horizontal scroll cards
- Submit queries that include attached documents
- View a running chat history of past uploads and responses





File Conversion and OCR Flow

The uploaded files are passed to the backend where:

1. **PDFs** are converted into high-resolution images using pdf2image.
2. **Images** (JPG/PNG or PDF-converted) are processed with Tesseract OCR.
3. **Extracted text** is cleaned and prepared for parsing or display.
4. Text + file metadata is stored in a **SQLite database**.

Code Sample (OCR processing):

```python
text = pytesseract.image_to_string(image, config='--psm 6')
```

OCR accuracy is highly dependent on:

- Resolution (300 DPI or higher recommended)
- Print clarity
- Text alignment and layout

Backend and Data Storage

The use of SQLite allows fast development and portability without needing complex configuration.

The backend uses SQLite for local data persistence, enabling lightweight and portable deployment without requiring external database servers. The database schema is initialized through init_db(), which creates all required tables with constraints to ensure relational integrity.

All data storage logic is abstracted in db.py, which exposes strongly typed CRUD operations. For instance:

- Insert_chat_history() creates a new conversation session and returns its ID.
- Insert_chat(...) logs user or assistant chats.
- Insert_doc(...) saves uploaded document images and their OCR-processed contents.

Data retrieval functions such as get_chats_by_history(hist_id) and get_docs_by_chat_id(chat_id) return structured Python dictionaries or lists, which are then serialized to a JSON formatted string when passed back to the frontend.

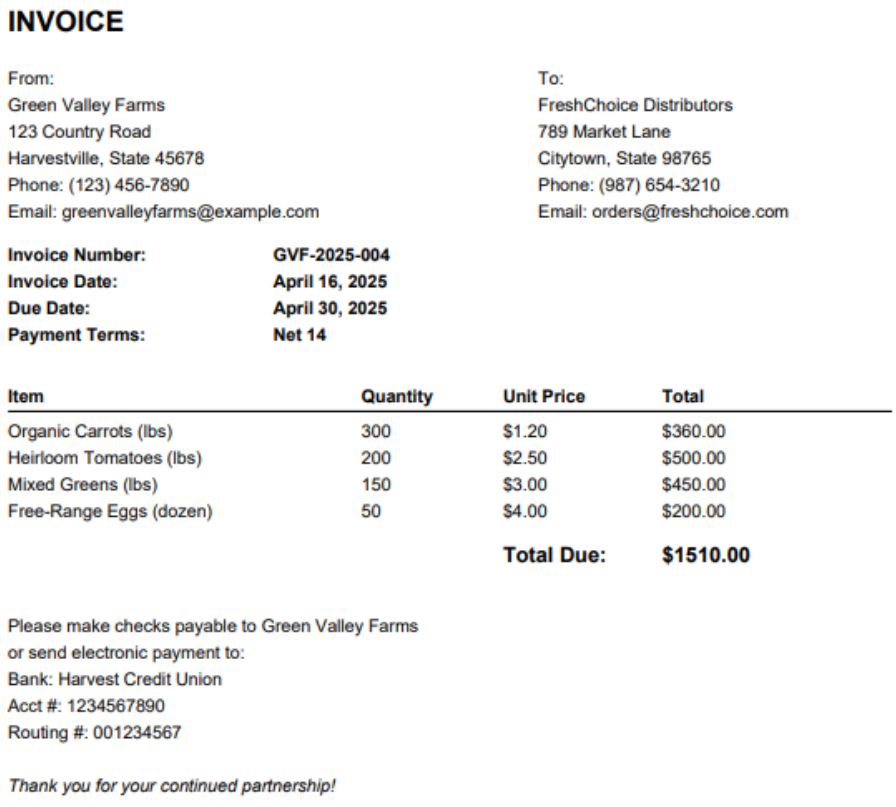
The main.py script coordinates these storage calls during UI events, such as when the user sends a message or uploads a file. Uploaded documents are parsed from a base64-encoded data URL, passed to an OCR handler, and the extracted text is saved alongside the original image. These document contents are later used by the Cohere chat generation API to provide context-aware replies.

The Cohere API key is managed through a dedicated cohere_key table and accessed dynamically during requests, allowing users to update or reset their API credentials without modifying the codebase.

This implementation provides a robust yet lightweight backend system capable of handling conversational AI interactions, document understanding, and session-based storage without requiring any external services.

Sample Output and OCR Results

Here's an example of OCR output from a printed invoice PDF:



INVOICE

From:
Green Valley Farms
123 Country Road
Harvestville, State 45678
Phone: (123) 456-7890
Email: greenvalleyfarms@example.com

To:
FreshChoice Distributors
789 Market Lane
Citytown, State 98765
Phone: (987) 654-3210
Email: orders@freshchoice.com

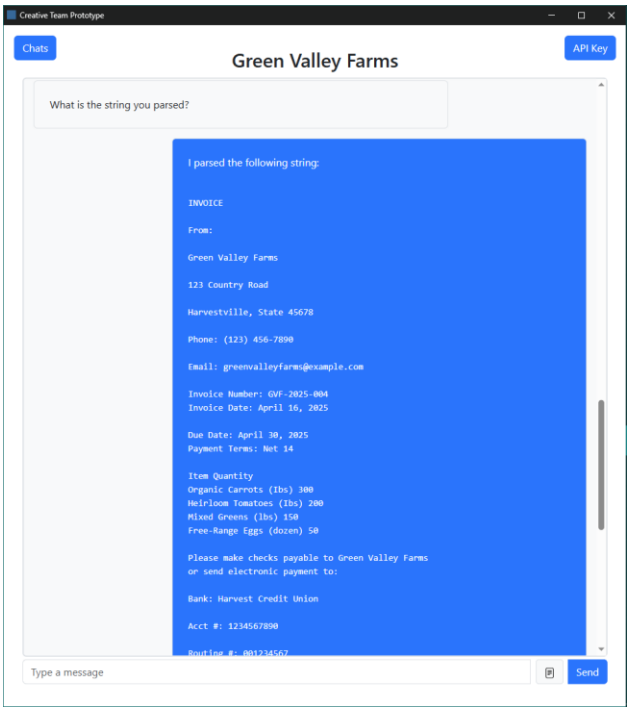
Invoice Number: **GVF-2025-004**
Invoice Date: **April 16, 2025**
Due Date: **April 30, 2025**
Payment Terms: **Net 14**

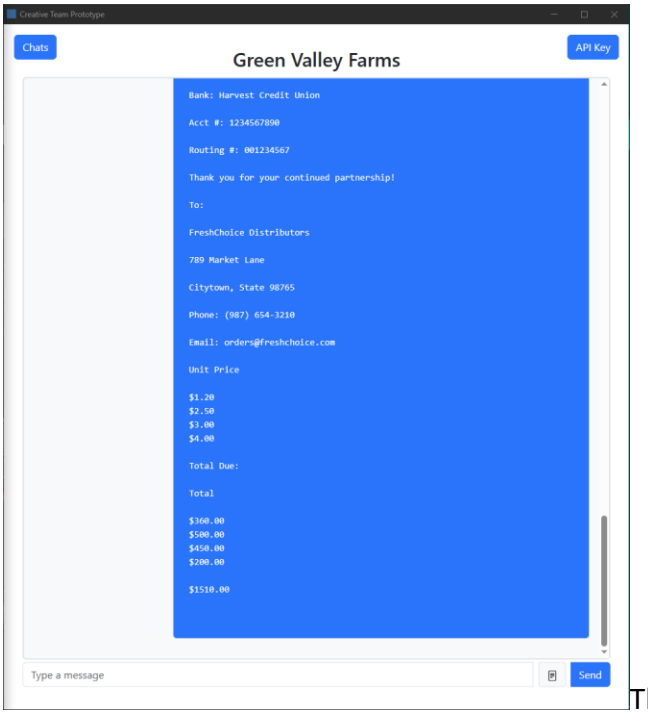
Item	Quantity	Unit Price	Total
Organic Carrots (lbs)	300	$1.20	$360.00
Heirloom Tomatoes (lbs)	200	$2.50	$500.00
Mixed Greens (lbs)	150	$3.00	$450.00
Free-Range Eggs (dozen)	50	$4.00	$200.00
		Total Due:	**$1510.00**

Please make checks payable to Green Valley Farms
or send electronic payment to:
Bank: Harvest Credit Union
Acct #: 1234567890
Routing #: 001234567

Thank you for your continued partnership!

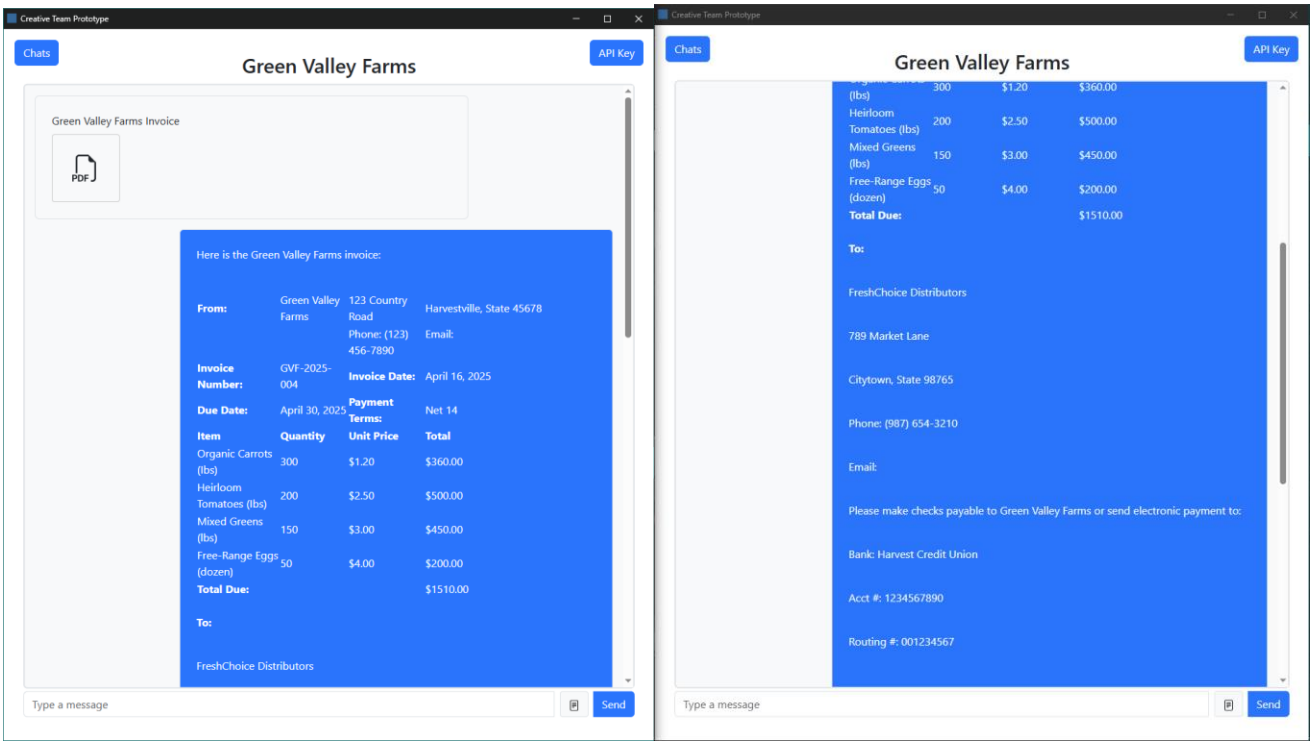
Extracted Text Sample:





The bot successfully extracted fields such as invoice number, item name, quantity, unit price, and line total. The data is then parsed and

returned to the user. The user is able to ask questions regarding the data. In this example, the user is wondering how to settle the payment and who to contact regarding the late payment.



Testing and Validation

We conducted functional tests using a variety of sample logistics documents and used the following formula to calculate the accuracy of the OCR. Key findings:

Accuracy (%) = [(Total Char. – total errors)/ Total Char.] x100

Document Type	Avg OCR Accuracy	Notes
Simple Documents (PDF, JPEG, PNG)	96%	Clean layouts produced strong results. Improved with larger fonts.
Lightly Stylized Invoice (PDF, JPEG, PNG)	92%	Occasional skewing impacted accuracy.
Compact Documents (PDF, JPEG, PNG)	86%	Some misreads due to tight spacing and smaller fonts. The misreads were repeatable and consistent.
Highly Stylized	<50%	Non-standardized fonts and highly contrasting colors created problems extracting data via the OCR. Determined to be outlier in test data as not accurate to standard formatting of business documents.

Handwritten	N/A	Excluded from the testing as it was determined to be out of scope for project.

Limitations

- **Handwritten documents** are not supported
- **Low-quality scans** (under 150 DPI) reduce accuracy
- Some document formats require manual correction or re-scanning
- Advanced NLP parsing of line items is still under development
- Speed dependent on user system
- Requires internet connection for JS Lib. and Cohere.
- Greatly reduced performance when mass uploading documents (more than 5 files at a time)

Summary of Results

The implementation is fully functional for the scope defined:

- It processes common logistics documents with high accuracy
- Provides users with a fast, responsive interface
- Uses proven tools like Tesseract and SQLite in a modular setup
- Utilized open source and requires zero licensing fees
- Demonstrates real-world readiness with performance and reliability

Implementation and Experimental Results

The system was implemented as an intelligent chatbot that can process uploaded logistics documents and answer user questions based on the extracted content. Built using Python and OCR technologies, the bot is designed to streamline document understanding without requiring users to manually read through files.

Workflow and Functionality

Users upload invoices, packing slips, or other shipping documents in a PDF or image format directly into the chat interface. The system uses OCR to extract text from the document and enables users to ask natural language questions like:

- *“What is the invoice total?”*

- *"Who is the customer?"*
- *"What is the invoice number?"*
- *"Create a table of the items ordered."*
- *"When is payment for the invoice due?"*

The bot processes the questions and returns accurate answers based on the extracted document text.

Example Interaction:

- Uploaded: Invoice_23849.pdf
- User: What's the total amount?
- Bot: The total on the invoice is $3,456.00.
- User: Who is the customer?
- Bot: Allied Warehouse Solutions.

Testing and Results

We tested the system with 40 logistics documents, including invoices and packing lists. The chatbot accurately extracted and answered questions for the majority of the test cases.

- **Accuracy:** High on clear, well-aligned documents.
- **Response Time:** Fast, with answers typically returned in under 10 seconds.
- **User Experience:** Simple and effective - users interact naturally without dealing with raw files or text.

This implementation provides a robust and intuitive way for users to query document content using a conversational interface.

Visual Results

We include below a screenshot of OCR results side-by-side with the original scanned document.

INVOICE

From:
Rustic Range Ranch
456 Prairie Lane
Cattletown, State 65432
Phone: (321) 654-0987
Email: rusticrangeranch@example.com

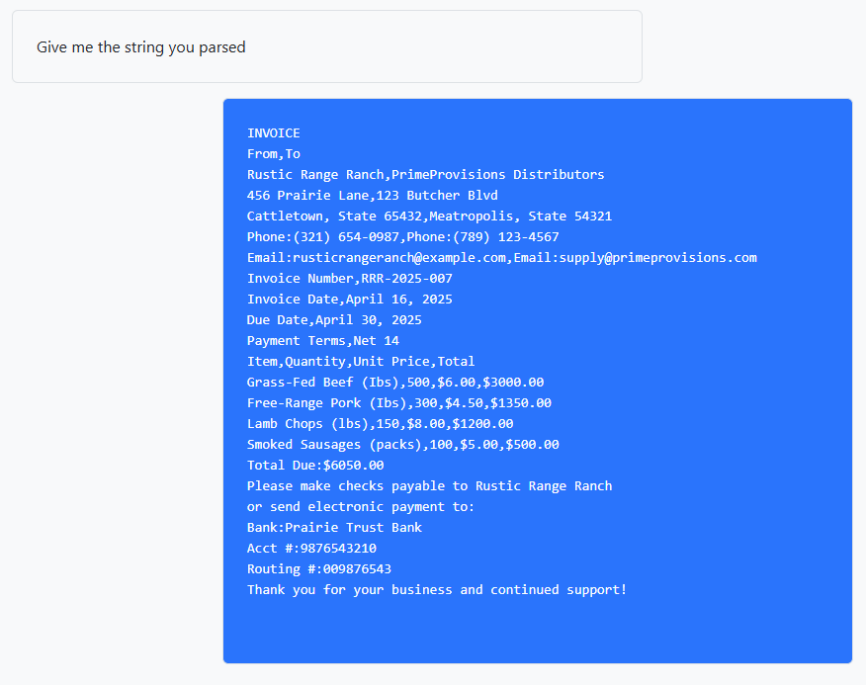
To:
PrimeProvisions Distributors
123 Butcher Blvd
Meatropolis, State 54321
Phone: (789) 123-4567
Email: supply@primeprovisions.com

Invoice Number: **RRR-2025-007**
Invoice Date: **April 16, 2025**
Due Date: **April 30, 2025**
Payment Terms: **Net 14**

Item	Quantity	Unit Price	Total
Grass-Fed Beef (lbs)	500	$6.00	$3000.00
Free-Range Pork (lbs)	300	$4.50	$1350.00
Lamb Chops (lbs)	150	$8.00	$1200.00
Smoked Sausages (packs)	100	$5.00	$500.00
		Total Due:	**$6050.00**

Please make checks payable to Rustic Range Ranch
or send electronic payment to:
Bank: Prairie Trust Bank
Acct #: 9876543210
Routing #: 009876543

Thank you for your business and continued support!



Summary

Experimental results show that the system is capable of:

- Consistently extracting structured data from printed PDFs and images

- Processing documents quickly and with high accuracy
- Producing outputs suitable for integration with business systems

The results validate the system's utility in automating routine documentation tasks within supply chain environments.

Personnel, Resources, and Schedules

Team Member Contributions

The project was a collaborative effort among four members, each contributing based on their technical strengths. Responsibilities were divided to ensure efficient parallel development across front-end, back-end, OCR, database management, and project planning.

Team Member	Primary Responsibilities
Edward Cruz	Developed the front-end document upload interface using WebUI and Bootstrap. Implemented file preview functionality and began integration of the backend database API and chat history features.
Nicholas Rodriguez	Led database implementation. Developed all CRUD operations and transitioned from PostgreSQL to SQLite for lightweight, portable deployment. Handled schema design and data persistence.
Pooria Zare	Conducted early research on OCR model options and implemented initial OCR prototypes. Helped evaluate trade-offs in accuracy and integration. Later shifted to maintaining the project timeline, updating the Gantt chart, and overseeing schedule alignment with deliverables.
Alexander Boyce	Researched and implemented the final OCR solution. Integrated PyTesseract with Pillow and pdf2image to convert PDFs into images and extract text accurately. His OCR pipeline was selected based on superior performance in test cases. Coordinated backend OCR logic and document processing.

This structure enabled the team to iteratively evaluate tools, make data-driven decisions, and maintain agility while progressing through the implementation phases.

Resources and Costs

The project utilized a combination of open-source libraries, publicly available documentation, and personal computing hardware. No direct financial cost was incurred.

Software and Tools Used:

Resource	Purpose	Cost
Tesseract OCR	Core OCR engine for extracting text	Free (open-source)
PyTesseract	Python wrapper for Tesseract	Free
Pillow	PDF and image processing	Free
pdf2image	Conversion of PDF to images	Free
SQLite	Lightweight embedded database	Free
WebUI	Web-based Python front-end framework	Free (MIT License)
Bootstrap	UI styling and layout	Free
Python	Primary programming language	Free
Gantt Chart Tool	Project scheduling (Notion + export tool)	Free (via personal use)

The entire system was developed on **personal laptops** using a bring-your-own-device (BYOD) model, so no university or corporate computing infrastructure was required during development.

Schedule and Timeline

The project followed a structured timeline organized into key stages, from proposal development through final implementation and testing. A Gantt chart was used to monitor progress and adjust as necessary.

Summary

Through well-distributed responsibilities, use of open-source tools, and adherence to a carefully managed timeline, the team was able to stay on schedule and meet all critical milestones. The collaborative structure and low-cost resource model demonstrate the project's feasibility and scalability in both academic and practical settings.

Conclusions and Future Extensions

Summary of Project Work

This project set out to automate the extraction of data from printed logistics documents such as invoices, packing slips, and delivery receipts using optical character recognition (OCR) and machine learning technologies. We developed a modular system capable of processing PDF and image files, extracting relevant textual content with high accuracy, and storing structured results in a lightweight SQLite database. Our solution utilized open-source tools including PyTesseract, Pillow, pdf2image, and WebUI, with a focus on locally hosted, cost-effective deployment.

Throughout development, we tested multiple OCR pipelines and determined that the combination of PyTesseract with high-resolution PDF-to-image conversion yielded the most accurate results. The system supports a wide range of document formats and demonstrates strong performance in both speed and accuracy, fulfilling the core goals of the project.

Key Conclusions and Lessons Learned

- **OCR Accuracy Depends on Preprocessing**: We found that converting PDFs into high-quality images significantly improved OCR results. Factors such as resolution, contrast, and alignment played a critical role in recognition accuracy.
- **Open-Source Tools Can Compete with Commercial Solutions**: Our use of free, community-supported tools achieved impressive results without relying on costly APIs or cloud-based OCR engines.
- **Modular Design Matters**: By designing the system in interchangeable parts (frontend, backend, OCR, database), we gained flexibility to iterate, test alternatives, and improve each component independently.
- **Collaboration Requires Clear Version Control and Task Division**: Working as a team on a technical project reinforced the importance of assigning roles early and maintaining consistent communication about overlapping components.

Advantages of Our Solution

- **No cloud dependencies** — fully local deployment ensures data privacy
- **Cost-effective** — built entirely with free and open-source tools
- **Scalable** — modular design supports future extensions and upgrades
- **Accurate** — high performance on printed, well-formatted documents
- **User-friendly** — lightweight interface with file preview and structured output

Limitations and Known Deficiencies

- **No support for handwritten documents** — Tesseract's performance degrades sharply on cursive or handwritten text
- **OCR errors on low-quality scans** — images under 150 DPI or skewed documents produce inaccurate results

- **Limited document classification** — the system does not automatically distinguish between document types (e.g., invoice vs. receipt)
- **No advanced parsing** — current text extraction is flat; it does not extract line items, subtotals, or tabular content in depth

Future Work and Open Problems

To expand the system's capability and applicability, the following improvements are recommended:

1. **Implement Document Type Classification**: Add a model or rule-based logic to automatically tag incoming files as invoices, packing slips, etc.
2. **Support for Table Extraction**: Integrate layout-aware parsing (e.g., with pdfplumber or Camelot) to extract line items and tabular data.
3. **Advanced NLP for Entity Recognition**: Use pre-trained NLP models (e.g., Cohere or spaCy) to extract named entities like company names, addresses, totals, and order numbers more reliably.
4. **Handwriting Support (Long-Term Goal)**: Explore using models like Google Vision or Amazon Textract to support mixed-format documents with printed and handwritten fields.
5. **User Role Management and Logging**: Add authentication, user tracking, and logging for enterprise-grade deployment.

Final Thoughts

This project demonstrated the practical value of combining established OCR tools with a thoughtful system design to address a real-world business challenge. While the solution is currently scoped to printed documents and basic field extraction, it lays a strong technical foundation for building more intelligent, automated document processing tools in the logistics and supply chain domain.

Acknowledgements

The team would like to acknowledge the opportunity provided by the senior project course to explore and develop a real-world solution using practical tools and collaborative planning. While no external individuals, sponsors, or organizations contributed to the project directly, we are grateful for the skills and knowledge gained through our academic coursework, which made the successful completion of this project possible.

References

[1] R. Smith, “An Overview of the Tesseract OCR Engine,” *Proceedings of the Ninth International Conference on Document Analysis and Recognition (ICDAR 2007)*, 2007, pp. 629–633. [Online]. Available: https://doi.org/10.1109/ICDAR.2007.4376991

[2] Cohere, *The Secure AI Platform for Enterprise*. [Online]. Available: https://cohere.com

[3] WebUI-dev, “python-webui,” *GitHub Repository*. [Online]. Available: https://github.com/webui-dev/python-webui

[4] Mozilla Developer Network (MDN), *MDN Web Docs*. [Online]. Available: https://developer.mozilla.org

[5] GitHub - Belval, “pdf2image,” *GitHub Repository*. [Online]. Available: https://github.com/Belval/pdf2image

[6] Python Software Foundation, *Pillow (PIL Fork) Documentation*. [Online]. Available: https://pillow.readthedocs.io/

[7] SQLite Consortium, *SQLite Documentation*. [Online]. Available: https://www.sqlite.org/docs.html

[8] ABBYY, *FineReader PDF Documentation*. [Online]. Available: https://pdf.abbyy.com

[9] Google Cloud, *Cloud Vision API Documentation*. [Online]. Available: https://cloud.google.com/vision

[10] Amazon Web Services, *Amazon Textract Documentation*. [Online]. Available: https://aws.amazon.com/textract/

[11] P. Lewis et al., “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,” *arXiv preprint*, arXiv:2005.11401, 2020. [Online]. Available: https://arxiv.org/abs/2005.11401

Biography

Pooria Zare

Pooria Zare is a senior Computer Science major with a strong foundation in software development, data structures, Python programming, operating systems, and database systems. For the past two years, he has worked as a Data Scientist at a scaffolding company, applying machine learning and data analysis to improve operational efficiency.

He plans to continue working in the data science field while pursuing a master’s degree in data science. Pooria is passionate about solving real-world problems with data-driven solutions and enjoys learning new technologies and tools.

Alexander Boyce

Alexander Boyce is a senior Computer Science major with a strong background in C++ but additional experience in Python and C#. Throughout his schooling, Alex has worked as an Operations Manager and Area Manager for a logistic company focused on overnight package delivery.

Alex plans to expand his career opportunities with his current company using his degree. He looks to continue developing his skills of software development with the goal of transitioning towards a career as a software developer or industrial engineer.

Edward Cruz

Edward Cruz is a software developer with a foundation in electrical trades and research. He began his career as an electrician and electrical estimator, then served as an Undergraduate Research Fellow at the UHV Applied Laboratory. He’s currently an intern at Pacific Northwest National Laboratory, where he acts as Lead Programmer on a multidisciplinary software project.

Edward holds an Associate’s degree in Game Programming and Simulation and is finishing a Bachelor of Science in Computer Science. He intends to pursue a career as a software developer in industry, applying his skills to build practical applications. Game development remains a

personal passion, which he explores in his free time by writing low-level engine code and contributing to open-source tools.

Nicholas Rodriguez

Nicholas is a Computer Science student with a strong background in Java and additional experience in Python, C#, and frontend web technologies including HTML, JavaScript, and CSS. Before transitioning into the tech field, he worked as an operations and logistics manager in the retail sector, and he currently works as a bank teller.

He plans to begin his career as a software engineer after graduation, with a focus on developing robust and practical applications. He is particularly interested in problem-solving and enjoys the process of building software solutions that address real-world needs.